Mini-Katran: A Stateless, Load-Aware Layer-4 Load Balancer

Akshit Chhabra *24110026*
Bhavay Goyal *24110070*
Devarshi Savalgi *24110103*
Kunal Chandola *24110176*

I. Introduction

A layer-4 (L4) load balancer sits in front of a group of servers and decides which server each connection goes to. It works on packets, not on requests, so it can pick a server only when a connection starts. After that, every packet of that connection has to reach the same server.

Well-known L4 load balancers such as Maglev [1] and Katran [2] give every server an equal or a fixed share of the traffic. This is fine when all servers are the same, but real clusters have older and newer machines, and a server can slow down when another job runs next to it. With fixed shares the slow server gets overloaded while the others still have room. Part of the cluster's capacity is wasted and the latency target is missed.

Fixing this means moving traffic away from a loaded server while the system is running. The hard part is to do it without breaking the connections that are already open, and without keeping a table of every connection on the load balancer. Cloudflare's Unimog [3] showed one way to do this. Our project, Mini-Katran, is a small version of that idea which we built ourselves and then measured. The name comes from the first version of our project, which copied Facebook's Katran [2]. The design in this report follows Unimog and not Katran.

In Mini-Katran, an eBPF program at the Linux TC hook hashes each packet into one of 4096 buckets and sends it to the server that the bucket points to. A controller reads a load signal from every server every 1–2 seconds and moves buckets from loaded servers to lighter ones. Each bucket also names a second server. A small program on every server, the redirector, forwards packets of older connections to that second server, so a connection keeps working after its bucket has moved. The load balancer keeps no state per connection.

We make three contributions.

First, a working system with the three parts above. It runs at TC and not at XDP, so it works on laptops that have no network card with native XDP.

Second, one rule that tunes the controller for every load signal from the noise we measured in that signal, so that comparing signals does not depend on how carefully we tuned each one.

Third, an evaluation on four servers of unequal speed, where we ask how much of the servers' total capacity each policy can use within a latency target (p99 $\leq$ 20 ms).

The table that the controller finds by itself from CPU utilisation uses 94.4% of the total capacity, against 94.7% for a table built from capacities we measured by hand and 64.9% for equal shares. When a server loses capacity, the controller is back within the target in 5–55 s. No connection broke in 91 live runs, while 36 to 45 long-lived connections broke per run when the redirector's forwarding was turned off.

We also report two results we did not expect: CPU utilisation worked better as a signal than requests in flight (94.4% against 87.3%), and a fixed table that already knows about the disturbance still does better than every controller.

II. Problem Statement

The problem is this. A group of servers with unequal and changing capacity sits behind one L4 load balancer. The load balancer can only choose where new connections go. It should carry as much load as possible within a latency and error target (p99 $\leq$ 20 ms, errors $\leq$ 0.1%), without breaking connections that are open and without storing anything per connection.

The inputs are the clients' TCP packets to one virtual IP address, and one load number per server (CPU utilisation or requests in flight), read every 1–2 s. The output is, for every packet, the server it is sent to. Behind that is a table of 4096 buckets that the controller keeps changing. We judge the result by the usable capacity fraction: the highest load that meets the target through the load balancer, divided by the sum of what each server can carry when it is tested alone.

We assume TCP over IPv4, one virtual IP and one dedicated load balancer. We assume that we control the servers, so we can run the redirector on each of them and read their load. The servers reply straight to the client, so the load balancer sees only one direction of a connection and cannot see requests or when a connection ends. The network between the load

balancer and the servers must have room for the 40 bytes that encapsulation adds.

Out of scope are UDP, IPv6, ICMP, fragments and IP options, more than two servers per bucket, automatic health checks and a second load balancer. We did not test what happens when a server fails. All main measurements are on one laptop with network namespaces and one synthetic workload. A single run on four real laptops is reported separately.

Existing systems each cover a part of this. Maglev [1] and Katran [2] spread connections with consistent hashing and equal or fixed weights, and do not react to load. Unimog [3] moves buckets using CPU utilisation, which is the design we follow, but its public description does not say how much of the servers' capacity this wins back, or how another signal would do. Prequal [4] balances on requests in flight and latency, but it chooses a server for every request from fresh probes, which an L4 load balancer cannot do.

In what we read, none of them measures three things that we do: the usable capacity fraction of each policy on servers of unequal speed, CPU against requests in flight inside the same bucket controller, and the number of connections broken while a controller keeps moving buckets, with the second-server forwarding on and off.

III. Project Objectives

We had four research questions, and each gives one objective that is checked with a number. Before measuring we also wrote down seven hypotheses (H1–H7). The main ones are named here, and Section VII gives the outcome of all seven.

O1 (RQ1, capacity). Measure the usable capacity fraction of equal shares, shares by core count, shares in proportion to each server's measured capacity (we call this table the oracle) and the tables that the controller finds. We expected shares by core count to be clearly below the oracle (H1), and the controller's table to match the oracle within about 3%, which is what our short runs can resolve (H2).

O2 (RQ4, connections). Count the connections that break while buckets move. The target is zero with the redirector forwarding. The same runs with forwarding turned off show how many it saves (H7).

O3 (RQ2, changing capacity). When one server loses capacity, measure the time until p99 is back within 20 ms and the share of 5-s windows over it, for the controller and for a fixed table, with disturbances that last from 2 s to 120 s. We expected the controller to help only when the disturbance is slower than its 1–2 s period and the lifetime of a connection (H3).

O4 (RQ3, signals). Compare CPU utilisation, requests in flight, cgroup CPU and pressure (PSI) by the capacity of the table each one finds, the recovery time and the number of bucket moves, and check which of them notice a noisy neighbour (H4–H6).

IV. Architecture and Design

Fig. 1 shows the system. It has three parts: the director on the load balancer, a redirector on every server, and a controller. The director and the redirector are eBPF programs in the kernel. The controller is a normal program that runs on the load balancer machine.

The main ideas are not ours. The table of buckets with two servers per bucket, the swap of a bucket's two servers and the control loop that moves buckets by load are from Unimog [3]. Forwarding the packets of older connections to a second server comes from Beamer [6], which Unimog also uses. The rendezvous hashing is from GLB [5], and requests in flight as a load signal from Prequal [4]. What we added is the director at TC, one tuning rule for all signals, the dwell and the caps in the controller, the SYN cookie check in the redirector, and the measurements.

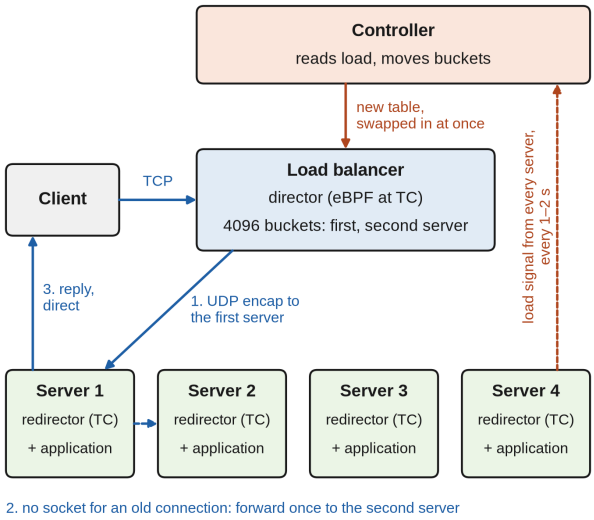


Fig. 1. Mini-Katran. Blue: the path of a packet. The dashed blue step happens only for a connection that was opened before its bucket moved; its reply then comes from the second server. Red: the control loop.

Packet path. The director looks at every packet that arrives at the load balancer. Packets that are not for the virtual IP go on to the kernel as usual. For a TCP packet to the virtual IP and port, the director hashes the 4-tuple (addresses and ports) with a fixed seed and takes the low 12 bits as the bucket number. The bucket's entry in the table names a first server and a second server.

The director then puts 40 bytes in front of the original IP packet: an outer IPv4 header addressed to the first server, a UDP header, and our own 12-byte header that carries the bucket number, the address of the second server and a flag. The packet goes back out of the interface it came in on. The original packet is not changed, so the server sees the client's own address and replies straight to the client. Replies never pass through the load balancer (direct server return).

Redirector. On the server, the redirector decides for every encapsulated packet whether to keep it or to forward it. It keeps the packet if one of these is true, checked in this order: the packet was already forwarded once (the flag is set); the bucket has no second server; the packet is a SYN, so it starts a new connection; the kernel has a socket for this connection (open, half-open or just closed); or only the listening socket matches, but the packet carries a valid SYN cookie, which means it completes a handshake that this server answered.

To keep a packet the redirector removes the 40 bytes, and the kernel handles the rest. Otherwise it sets the flag, changes the outer destination to the second server and sends the packet on, still encapsulated. So a packet is forwarded at most once.

This is what keeps connections alive. When a bucket gets a new first server, the old one stays in the bucket as its second server. Packets of older connections arrive at the new first server, which has no socket for them, and are passed on to the old one, which has.

Controller. Every period the controller reads one number u for each server, its load relative to its capacity, and computes the average ū, weighted by the servers' core counts. It does nothing while ū is under 0.2, because the signals of idle servers are mostly noise. It also does nothing inside a dead band: it starts to move buckets only when some server is further than δ_{on} from the average ($|u/\bar{u} - 1| > \delta_{on}$), and stops when every server is closer than δ_{off}, which is half of δ_{on}.

When it acts, a server that has n buckets gets the target $n \cdot (\bar{u}/u)^{0.3}$, limited to 25% up or down and to at least 16 buckets. Two caps limit what one period can do: a server gains or loses at most 40 buckets (1% of the table), and at most 81 buckets (2%) change in total.

The controller only touches buckets whose first server has not changed for 20 s (the dwell), the oldest first, and picks them in two passes. The first pass makes swaps: if a bucket's first server should lose buckets and its second server should gain, the two change places. The second pass makes reassignments for what is left: the bucket gets a new first server, the old first server becomes the second, and the old second server drops out of the bucket.

Publishing a table. The director does not hold the table itself. It reads it through a map with a single slot that points to the table (a map-in-map). The controller writes a complete new table of 4096 entries and then updates that one slot. A packet therefore sees either the old table or the new one, never a half-written mix. The first table comes from rendezvous hashing, as in GLB [5]: every bucket ranks the servers by a hash of the bucket and the server. Its first server is the highest-ranked one that still has room in its share, and its second server is the highest-ranked other one.

Tuning. The period P and the dead band are set by one rule for every signal. We measure the noise σ of the signal under a fixed table, averaged over P = 1, 2, 4 and 8 s, and set $\delta_{on} = k \cdot \sigma$ with k = 1.5, 2 or 3. A simulator of the controller runs every pair of P and k. Among the pairs that stay quiet when nothing is wrong (at most 40 moves per minute, which is 1% of the table, or twice the least that signal can reach), we take the one that recovers fastest after a server loses half its capacity.

For CPU utilisation this gave a period of 2 s and a band of 17%, and for total requests in flight 2 s and 48%. The gain, the caps, the floor and the dwell are the same for all signals. A noisier signal gets a wider band or a longer period, and that is then a property of the signal and not of our effort.

Why buckets and no connection table. The state on the load balancer is 4096 entries however many connections there are, and the director only reads it. The price is that every server has to run the redirector, and that load moves only as old connections end and new ones start. We also use one dedicated load balancer, where Unimog runs one on every server. Forwarding work on the servers would mix into the CPU numbers that we use both as the load signal and to measure capacity.

Why TC and not XDP. None of our laptops has a network card with native XDP. There XDP would run in generic mode, which starts after the kernel has built its packet buffer, the same point as TC, so it would not be faster. At TC the kernel still handles large segmented packets and unfinished checksums correctly after we encapsulate, and tcpdump sees what the load balancer sends. We give up the speed of native XDP, which we do not need at our packet rates. On virtual interfaces XDP cost 1.24 times the CPU time of TC per packet (Section VII).

Why swaps first, and the dwell. A swap keeps both servers in the bucket, so no connection can lose its server, however often swaps happen. A reassignment drops the old second server, which may still hold connections from an earlier move. The dwell is there for this case: a bucket is not touched again until the connections of its previous first server have ended. 20 s is far longer than the short connections of our workload (about 2.2 s), but a connection that lives longer can break in a reassignment. Beamer [6] avoids this with longer chains and generation numbers. We stayed with two servers per bucket because it is simpler and swaps cover almost every move.

Why rendezvous second servers. Because every bucket has a second server from the first table on, swaps are possible from the start. If the second server were only filled in after a move, the first move of every bucket would have to be a reassignment.

Why the SYN cookie check. Under load a server's listen queue overflows and the kernel answers with SYN cookies. The last ACK of such a handshake matches no socket yet. Without the check the redirector would forward it to the wrong server and break a new connection, exactly when we measure capacity.

V. Implementation

The director and the redirector are written in C and compiled to eBPF with clang. Both attach at TC ingress through TCX, which needs Linux 6.6 or newer.

Everything in user space is written in Go 1.27: the loader that attaches the programs and writes tables (lbctl), the controller, the readers for the load signals, the test server, the load generator, the noisy neighbour, a monitor that records every TCP reset, and a simulator of the controller. The only Go library is cilium/ebpf v0.22.0. It loads the programs, fills the maps, and also runs the programs on prepared packets inside the kernel, which we use to test the director and the redirector byte by byte.

The experiments are driven by Python 3 scripts and plotted with matplotlib. No special hardware is needed (Section VI). The code is at https://github.com/akshit-codes/CS331-T020-MiniKatran.

All load signals are counters that the controller reads once per period and takes the difference of. CPU utilisation is the busy time of the server's cores from /proc/stat. Requests in flight come from a counter that the server application exports. Cgroup CPU and pressure (PSI) come from the server's cgroup files.

The test server is a Go HTTP server in which every request does a fixed amount of work, not a fixed time, so a slower core really takes longer. It has one worker thread pinned to each of its CPUs and one queue. The load generator is open loop: it sends at the scheduled times whether or not earlier replies have arrived, and counts latency from the scheduled time.

Algorithm 1 is one period of the controller, with the constants we used.

Algorithm 1: one control period

```
u[i] = load of server i, from the signal
m = sum(c[i]*u[i]) / sum(c[i])      (c = cores)
if m < 0.2: stop
dev = largest |u[i]/m - 1|
if not active and dev > delta_on: active = yes
if active and dev < delta_off:    active = no
if not active: stop
for each server i, with n[i] buckets:
    t[i] = n[i] * (m / max(u[i], 0.05))^0.3
    keep t[i] within 25% of n[i], and >= 16
scale t so that it sums to 4096
need[i] = t[i] - n[i], at most 40 either way
S = buckets unchanged for 20 s, oldest first
for b in S:                          (swaps)
    if need[first(b)] < 0 < need[second(b)]:
        exchange first(b) and second(b)
for b in S, not yet moved:           (reassign)
    r = server with the largest need > 0
    if need[first(b)] < 0 and r exists:
        second(b) = first(b); first(b) = r
stop either loop after 81 moves in total
publish the new table with one map update
```

Four problems cost us the most time. The first was a race when the controller starts. It published its first table over the table that was live, without second servers, at the moment the load generators started. A connection opened just before that could end up on a server that was no longer in its bucket. We never saw it happen (0 resets in the 34 runs before the fix), we found it when reviewing the code.

Now the first table is built from the live one: a bucket whose first server changes keeps the old one as its second server and counts as just changed, so the dwell protects it. While fixing this we found that our unit test for the dwell still passed with the dwell check removed, and wrote a test that fails without it.

The second was the SYN cookie case of Section IV. We had expected it only under overload. A test that moved half of the buckets showed it with no overload at all: with the check turned off, 632–671 of about 5,950 new connections broke.

The third was the neighbour for hypothesis H6, which runs on the idle hyperthreads of a server's cores. Our first neighbour was the request loop itself, and it did not slow the server at all (signals at 0.96–0.97 of their earlier value). We built one that walks a 64 MiB buffer and so pushes the server's data out of the caches the two threads share. With it CPU time per request rose by about 10%.

The fourth came from a teammate's Ubuntu laptop. clang 21 writes the type of a table entry in a nested map only as a forward declaration, and the loader could then not tell its size and refused the director. We now write the key and value sizes out. Built with one compiler, the old and the new source give the same instructions and map sizes, so no run had to be repeated.

VI. Testbed Setup

Machine. Everything runs on one laptop with an Intel Core i7-14650HX (8 P-cores with two threads each and 8 E-cores, 24 CPUs) and 32 GB of RAM, on mains power. The software is Omarchy 4.5 (based on Arch Linux) with Linux 7.2, clang 23, Go 1.27 and Python 3.14.

Topology. The testbed is seven network namespaces joined by virtual Ethernet pairs. The client (cli, 10.0.0.2) connects to a router (rtr). The router forwards between the client and a bridge, and the load balancer (lb, 10.1.0.10) and the four servers (10.1.0.11–14) hang on that bridge. The router sends packets for the virtual IP (10.2.0.1) to the load balancer. Every server has the virtual IP on its loopback interface and does not answer ARP for it, so it accepts the clients' packets and replies directly. The MTU is 1500 on the client side and 1600 between the load balancer and the servers.

CPUs. Each role has its own CPUs. CPUs 0–1 are for the desktop and our scripts, 2–5 for the load generator, 6 for the router and the load balancer, and Table I gives the servers'. The second threads of the servers' P-cores stay idle. Receive packet steering makes each role process its own packets on its own CPUs. Before a session, scripts/tune.sh turns turbo off,

fixes the clock speed, turns deep idle states off, confines the desktop and system services to CPUs 0–1 and moves device interrupts there. It writes nothing to disk and undoes everything afterwards.

TABLE I. THE FOUR SERVERS

Server	CPUs	Cores	Capacity alone (req/s)	Share in the oracle table
beP1	8, 10	2 P, slowed 1.6×	957	15.4%
beP2	12, 14	2 P	1656	26.6%
beE1	16–19	4 E	1810	29.0%
beE2	20–23	4 E	1810	29.0%

Workload. A request costs 1 ms of CPU on a P-core on average, exponentially distributed, and 1.85 times as long on an E-core. The cost is calibrated at the start of every session. Requests arrive as a Poisson process over 512 connections. A connection carries 20 requests and is then replaced by a new one, so it lives about 2.2 s at 75% load. Runs with a controller add 100 long-lived connections with 2 requests per second each, which stay open for the whole run and so outlive the dwell.

Target and capacity. A load passes if p99 ≤ 20 ms, errors ≤ 0.1% and at least 98% of the scheduled requests are served. A probe holds one rate for 5 s of warm-up, 20 s of measurement and 2 s of rest (27 s). A capacity is the highest passing rate found by bisection over 5 probes, with a resolution of 1–2%. Table I gives each server's capacity alone, without the load balancer. Their sum, 6234 req/s, is the denominator of every usable capacity fraction.

Disturbances. These runs hold the load at 75% of that sum and record p99 in 5-s windows. In the step, server beP2 needs twice the CPU per request from 60 s on. In the antagonist run, a neighbour in another cgroup uses 60% of the CPU time of beE1's cores from 60 s to 180 s. A run has recovered when p99 is within the target for three windows in a row.

Four laptops. One run used real machines over a phone hotspot: a client, a load balancer and two laptops with two servers each. The servers stay in namespaces behind their laptop, which routes to them, and the controller reads the signals over the network. Because of the wifi, a request costs 40 ms, the target is p99 ≤ 800 ms, and there are 32 connections and 20 long-lived ones. Each configuration ran once.

VII. Evaluation

How to read the numbers. All results are from the single laptop and one synthetic workload, except the run in part E. Every result says how many runs it is from. The main comparisons have 5 or more runs, taken in one session in interleaved order (A B C, B C A, and so on), so that a slow drift of the machine does not favour one policy. A value after ± is the 95% interval of the mean (Student's t). We call a difference resolved when its 95% interval does not include zero. With 5 runs that is evidence, not certainty. Where there are only 1–4 runs we trust only differences larger than about 5%.

Two table names promise more than they are. The oracle is the table in proportion to each server's capacity measured alone, before any disturbance. Nobody searched for the best split and it is not one. Hindsight is a table in proportion to the capacities during one known disturbance, measured beforehand, which no real system has.

Finally, the repeat runs were made in one night in which the servers alone were 3.6% slower than in the first sessions (their sum was 6011 against 6234 req/s, and we do not know why). We keep 6234 as the denominator everywhere. The differences between tables do not depend on it.

A. Capacity (RQ1)

We searched for the capacity of five tables through the load balancer, five interleaved runs for the three main ones (Table II, Fig. 2). The two controller tables were found new in every run: the controller starts from shares by cores, runs for 90 s at 75% load, and the table is then frozen.

TABLE II. USABLE CAPACITY FRACTION THROUGH THE LOAD BALANCER (S8)

Table	Runs	Mean ± 95%	Range
Equal shares	2	64.9%	64.9–64.9%
Shares by cores	2	86.3%	84.7–88.0%
Oracle	5	94.7% ± 2.4	91.5–96.2%
Found by CPU controller	5	94.4% ± 2.2	92.1–96.3%
Found by total-RIF controller	5	87.3% ± 4.2	84.0–92.9%
Difference (5 runs each)	**Points**	**95% interval**	**Resolved**
Oracle − CPU	+0.3	−2.5 to +3.1	no
Oracle − total RIF	+7.4	+3.1 to +11.7	yes
CPU − total RIF	+7.1	+2.7 to +11.5	yes

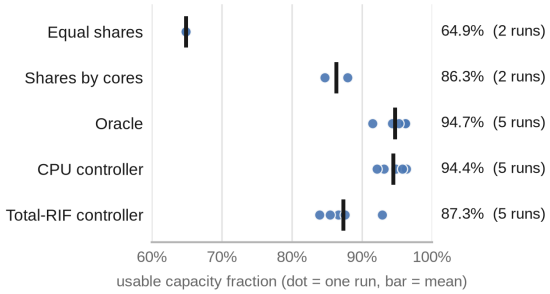


Fig. 2. Usable capacity fraction of each table (S8). The two controller tables were found new in every run.

The CPU controller is never told a capacity, and its table reaches 94.4% against 94.7% for the oracle. The interval of the difference says that the CPU table is at most about 3 points worse (H2 holds for CPU). Shares by cores lose 8.4 points to the oracle and equal shares about 30, because an E-core is not worth a P-core (H1).

The total-RIF controller did not do as well. Its tables reach 87.3%, which is 7.1 points below the CPU tables, and four of its five runs are below every CPU run. This corrects our own earlier result: with 3 runs each we had 92.7% against 90.9% and wrote that the two signals could not be told apart.

The tables show why. Total RIF steers every server towards the same number of requests in flight. The same number is a longer queue on a server with 2 cores than on one with 4, so the 2-core servers get more than their share (beP1 gets 17.5–18.6% of the buckets, against 15.4% in the oracle) and miss the target first. We found this explanation after the data and did not test it separately. It is a property of the signal and not a fault of the director or the controller.

A capacity search turns a noisy p99 into pass or fail near 20 ms. As a check without a threshold, we ran each table for 60 s at two fixed loads, again five interleaved rounds (Table III). The order is the same. At 80% load every difference from the oracle is resolved, also the CPU table's half a millisecond, which the search could not see. At 90% load the total-RIF tables miss the target in 2 of 5 rounds and shares by cores in all 5.

TABLE III. P99 AT FIXED LOADS, 5 RUNS EACH, MEAN ± 95% (S14)

Table	p99 at 80% load (ms)	p99 at 90% load (ms)	Runs within target at 90%
Oracle	10.6 ± 0.1	13.8 ± 0.9	5 of 5
CPU controller	11.1 ± 0.4	14.9 ± 1.0	5 of 5
Total-RIF controller	12.7 ± 0.5	21.3 ± 7.0	3 of 5
Shares by cores	13.6 ± 0.5	34.0 ± 10.9	0 of 5

Why does even the oracle stop near 95%? The same shares sent straight to the servers, without the load balancer, reach 100.2% ± 1.3, against 96.8% ± 1.0 through it (5 runs each, from two sessions). The gap of 3.4 points (+2.0 to +4.8) is resolved. Through the load balancer a whole connection goes to one server, and the requests arrive less evenly (the variation of each server's requests per second is 4.9–8.6%, against 1.6–4.3% without it). That fits the gap but does not prove it, because the two cases differ in several ways that we did not separate.

B. Connections (RQ4)

With the redirectors forwarding, no long-lived connection broke and no server sent a reset in 91 live runs. 34 of them are from before the startup fix of Section V, with about 75,000 bucket moves. 57 are on the final code, with about 69,500 moves and 0 of 5,700 long-lived connections broken. In the 53 runs of the repeat night the controllers made 63,310 moves (62,993 swaps and 317 reassignments) while 37.2 million short requests passed with 0 errors. With 0 of 5,300 connections broken, the 95% upper bound is about 0.06% per connection.

The same scenario with forwarding turned off broke 36, 45 and 38 of the 100 long-lived connections (3 runs), and 160–227 short ones. All moves in those runs were swaps, so a swap is safe only because the second server forwards. In an earlier test that moved half of the buckets at once, 0 of 300 long-lived connections broke with second servers and 130–146 of 300 without (2 runs).

One result here went against what we first thought. A controller without the dwell and the caps made 16,311 moves in its first run and broke nothing, because all of them happened to be swaps. Two more runs gave a different picture: 643 reassignments broke 10 connections, and 15 reassignments broke 1. So the first 0 was luck, and the dwell is what covers reassignments. This is 3 runs, so the size of the effect is loose. We also have no live numbers for changes that need many reassignments, such as taking a server out.

C. Changing capacity (RQ2)

Table IV and Fig. 3 give the step and the antagonist at 75% load, with the fixed tables and the controllers interleaved in the same two sessions.

TABLE IV. STEP AND ANTAGONIST AT 75% LOAD (S9, S12). RECOVERY TIME (MEDIAN) AND SHARE OF 5-S WINDOWS OVER THE TARGET

Table (runs per scenario)	Step: recovery	Step: over	Antagonist: recovery	Antag.: over
Oracle (2)	none, 12–14% errors	79–83%	never	100%
Hindsight (2)	0 s	0–8%	0–10 s	0–4%
CPU controller (5)	30–55 s (35)	25–46%	15–20 s (15)	12–25%
Total-RIF controller (5)	30–40 s (40)	29–58%	10–30 s (15)	8–25%

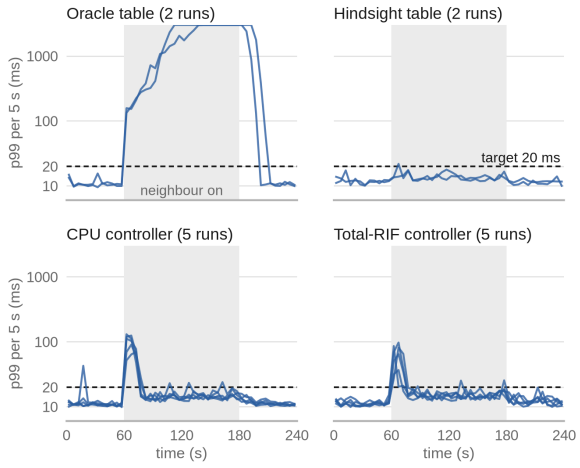


Fig. 3. p99 in 5-s windows in the antagonist runs of S9 and S12, one line per run, log scale. The neighbour runs on beE1's cores in the grey band. Lines at the top edge are 3 s or more.

The controllers get back within the target by themselves. Over all sessions (7–10 runs per controller and scenario) recovery takes 5–55 s. The oracle table does not recover: after the step 12–14% of the requests fail, and under the antagonist every window is over the target until the neighbour stops.

After recovery the CPU controller's p99 is 15.0–17.0 ms on the step and 13.6–16.5 ms under the antagonist.

But the hindsight table does better than every controller: no transient, no moves, and at most 8% of the windows over the target, against 8–58% for the controllers. So we cannot say that the controller beats fixed tables. It beats a table that is out of date. What the controller gives is that nobody has to know which server will slow down, when, or by how much. A hindsight table for one disturbance does nothing for a different one.

The limit is load, not speed. We repeated the scenarios at other loads with 1 run each (Table V). At 60% the picture is the same with shorter recoveries. At 85% load, which is 94% of the capacity that the antagonist leaves, nothing held the target: the hindsight table was over it in 58% of the windows, the CPU controller in 100% and total RIF in 88%. The controllers still brought p99 down from 14.7 s (oracle) to 22–29 ms, just over the target. A controller can only spread load over capacity that exists.

TABLE V. OTHER LOADS (S13), 1 RUN PER CELL. RECOVERY TIME; SHARE OF 5-S WINDOWS OVER THE TARGET

Scenario and load	Oracle	Hindsight	CPU controller	Total-RIF controller
Antagonist at 60%	0 s; 21%	0 s; 0%	5 s; 4%	0 s; 0%
Step at 60%	never; 100%	0 s; 0%	15 s; 12%	20 s; 17%
Antagonist at 85%	never; 100%	5 s; 58%	never; 100%	never; 88%

Table VI varies how fast the neighbour switches on and off. H3 expected the controller to stop helping once the disturbance is faster than the control period and the connection lifetime (about 2.2 s). That did not happen. With a neighbour that switches every second, the controllers still cut the windows over the target from 100% to 12–21%. They do not follow the switching. They see its average as a server that is slower all the time and move load once. Most points are single runs, so only the large gap between the oracle and the controllers counts, and no hindsight table was run here.

TABLE VI. SHARE OF WINDOWS OVER THE TARGET WHEN THE NEIGHBOUR SWITCHES ON AND OFF (RUNS IN BRACKETS)

Period (on for half of it)	2 s	5 s	20 s	60 s
Oracle	100% (1)	96% (1)	50% (2)	50% (1)
CPU controller	21% (2)	12% (1)	15% (2)	8% (1)
Total-RIF controller	17% (2)	12% (1)	12% (1)	12% (1)

After the step the CPU controller settles (0–143 moves per minute), but total RIF keeps moving buckets back and forth (756–1046 per minute). All of these moves were swaps and p99 stayed within the target, so nothing broke, but our simulator had predicted at most 40 moves per minute, so it underestimates the noise of the live system. Changing the gain from 0.1 to 1 made no difference that 2 runs per setting could show.

D. Signals (RQ3)

CPU utilisation was the best signal here. It finds the better table (part A) and it settles. Under a disturbance CPU and total RIF recover alike: the median is 12.5 against 10 s under the antagonist (10 and 9 runs) and 32.5 against 40 s after the step (8 and 7 runs), with overlapping ranges.

H4 had expected the opposite, that CPU would lose to requests in flight on unequal cores. Dividing requests in flight by the core count, which was our first choice, loses capacity in the other direction (86.6%, 2 runs) and moves the most buckets. This is a result for our controller and our four servers and not a test of Prequal [4], which picks a server for every request.

Cgroup CPU does not see a neighbour in another cgroup: its value even fell to 0.91 times, while whole-core CPU rose to 1.32 times (1 run). A controller on this signal never recovered in 3 runs (H5).

PSI sees request overload only if the server queues in the kernel. With our worker-pool server it stayed at or under 0.14 even at 130% load; with a server that queues in the kernel it rose to 0.68 at 90% load and 0.99 at 130% (1 run per mode).

For H6 we put the cache-walking neighbour on the idle hyperthreads of beP1. CPU rose 1.10 times, cgroup CPU 1.09, requests in flight 1.25 and PSI not at all (1 run). So "only requests in flight notice" is wrong, but every rise is inside that signal's dead band and no controller would have acted on it.

E. Other measurements

On virtual interfaces the director costs less CPU at TC than at XDP: XDP took 1.24 ± 0.06 times TC's time per packet at 2000 req/s and 1.25 ± 0.09 at 4000 (4 repeats in alternating order; the time includes the router on the same CPU). This says nothing about network cards with native XDP. Below the knee the load balancer adds almost nothing to latency: through it, p99 was 0.07 ± 0.20 ms higher at 50% load and 0.04 ± 0.28 ms at 80% than with the same shares sent directly (5 pairs, not resolved). Its cost shows only near capacity (part A).

The run on four laptops has 1 run per configuration and cannot be compared with the numbers above. With forwarding on, 0 of 20 long-lived connections broke through 1223 and 511 bucket moves; with it off, 12 of 20 broke. Starting from equal shares, the CPU controller reached shares within 1.7 points of the oracle's. Under the antagonist it held p99 lower than the fixed table (2.9 s against 12.9 s), but nothing got back under the 800 ms target. The capacities cannot be ranked, because one table measured twice differed by about 3 times in p99 at the highest rate.

F. Hypotheses

Table VII gives the outcome of the seven hypotheses we wrote down before measuring.

TABLE VII. OUTCOME OF THE HYPOTHESES

Hypothesis (short)	Outcome

H1	Shares by cores are below the oracle	Supported: 86.3% against 94.7%
H2	The controller's table matches the oracle and breaks nothing	CPU: supported (0.3 points below). Total RIF: not supported (7.4 points below)
H3	The controller helps only for slow disturbances	First half holds. Second half not supported (it helped at a 2-s period)
H4	CPU loses to requests in flight on unequal cores	Not supported: the reverse holds for the tables
H5	Cgroup CPU is blind to a neighbour; PSI and CPU see it	Supported (1 signal run, 3 controller runs)
H6	Only requests in flight see a neighbour on the sibling threads	Not supported: CPU signals see it too, PSI does not (1 run)
H7	One change breaks nothing; breaks grow with the change rate	First half supported (0 broken in 91 runs). Second half weakly (11 broken in 3 runs without dwell and caps)

VIII. Summary

An L4 load balancer can only choose a server when a connection starts. With fixed shares it wastes capacity when the servers are unequal, and it misses the latency target when one of them slows down. Mini-Katran follows the design of Cloudflare's Unimog [3]. It spreads connections with a table of 4096 buckets, which a controller keeps adjusting from a load signal. Every server forwards packets of older connections once to the bucket's second server, so connections survive when buckets move, and the load balancer keeps no state per connection.

On four servers of unequal speed the table that the controller finds by itself from CPU utilisation uses 94.4% of the servers' total capacity. That is within about 3 points of a table built from measured capacities, and well above shares by core count (86.3%) and equal shares (64.9%). No connection broke in 91 live runs, and connections did break in every run with forwarding turned off. When a server lost capacity, the controller was back within the target in 5–55 s without being told anything.

Three results went against what we expected. CPU utilisation was a better signal than requests in flight, by about 7 points of capacity. A fixed table that already knows the disturbance does better than every controller, so what the controller gives is that nobody has to know the disturbance. And at 85% load nothing held the target, because a controller can only spread load over capacity that exists.

Limitations. All main results are from one laptop with network namespaces, which is not a real network, and from one synthetic workload. Some points are single runs: most of the timescale sweep, the other load levels, and the cgroup CPU, PSI and hyperthread measurements. The hindsight table was not run with a neighbour that switches on and off. The run on four laptops is one run per configuration and ranks nothing.

We did not test what happens when a server fails. From reading the code, and not from a run: with the CPU signal a dead server's cores look idle, so the controller would give it more buckets. A connection that lives longer than the 20-s dwell is safe only as long as its bucket is moved by swaps.

Future work. (1) Run the director on a network card with native XDP and compare its cost per packet with TC there.

(2) Add a health check, then drain and kill a server under load and count the broken connections, with connections that outlive the dwell.

(3) Replay a real workload with mixed request costs and long connections.

(4) Try a signal that mixes the two: CPU utilisation to find the table, and requests in flight to see overload, where CPU stops at 100%.

(5) Allow more than two servers per bucket, with generation numbers as in Beamer [6], so that a bucket can move again before the dwell ends.

(6) Repeat the single-run points and the four-laptop run, and run the hindsight table against the switching neighbour.

IX. Reproducibility

The code, the run records behind every number and this report are at https://github.com/akshit-codes/CS331-T020-MiniKatran. To run it you need Linux 6.6 or newer with BTF, cgroup v2 and systemd, and clang, the libbpf headers, Go, Python 3 with matplotlib, iproute2, ethtool, curl and sudo. Everything runs in network namespaces on one machine. The CPU numbers of our laptop are in code/config.env and code/backends.conf and have to be changed for another machine.

Commands, from the code/ folder

```
make
make test && make test-bpf
sudo scripts/topology.sh up
sudo scripts/topology.sh check
sudo scripts/topology.sh down
sudo python3 bench/e2e_lb.py
R=results/campaign-$(date +%Y%m%d)
sudo python3 bench/campaign.py s1 --root $R
python3 bench/tune_signals.py $R
S="s2 s2-extra s2-rif-total s3 s4 s5 s6 s7"
for s in $S; do
  sudo python3 bench/campaign.py $s --root $R
done
sudo bench/repeats.sh $R
python3 bench/analyze_campaign.py $R
```

The first two lines build the eBPF programs and the Go tools and run the unit tests and the packet tests in the kernel. The three topology lines only bring the testbed up by hand, check it and take it down again. They are not needed for the campaign, because every session applies scripts/tune.sh, builds the testbed, calibrates the request cost and undoes everything at the end.

The end-to-end check should pass before anything is measured. Session s1 measures the servers alone and the noise of the signals, tune_signals.py applies the tuning rule, and the other sessions are the experiments of Section VII. Adding --dry to a session runs a short version of it first. The last line writes $R/summary.md and all plots.

Sessions s1 to s7 took about 7 hours of machine time on our laptop, and the repeat sessions about 7 hours more. The laptop feels slow while a session runs. repeats.sh can be stopped at any time: it records every finished run, and when it is started again it continues with the runs that are left. If a run is killed hard, sudo scripts/tune.sh undo restores the machine.

All results in this report are in the folder code/results/campaign-20261102/ (summary.md, the plots, and one folder per session with its runs.jsonl). The four-laptop run is in code/results/cluster-20261106-085033/, and code/docs/cluster-runbook.md says how to repeat it. The scripts for the figures of this report are in report/figures/ (fig1.py, fig2.py, fig3.py).

References

[1] D. E. Eisenbud, C. Yi, C. Contavalli, C. Smith, R. Kononov, E. Mann-Hielscher, A. Cilingiroglu, B. Cheyney, W. Shang, and J. D. Hosein, “Maglev: A fast and reliable software network load balancer,” in Proc. 13th USENIX Symp. Networked Systems Design and Implementation (NSDI 16), Santa Clara, CA, USA, 2016, pp. 523–535.

[2] N. Shirokov and R. Dasineni, “Open-sourcing Katran, a scalable network load balancer,” Facebook Engineering, May 22, 2018. [Online]. Available: https://engineering.fb.com/2018/05/22/open-source/open-sourcing-katran-a-scalable-network-load-balancer/. Source code: https://github.com/facebookincubator/katran

[3] Cloudflare, “Unimog - Cloudflare’s edge load balancer,” The Cloudflare Blog, Sep. 9, 2020. [Online]. Available: https://blog.cloudflare.com/unimog-cloudflares-edge-load-balancer/

[4] B. Wydrowski, R. Kleinberg, S. M. Rumble, and A. Archer, “Load is not what you should balance: Introducing Prequal,” in Proc. 21st USENIX Symp. Networked Systems Design and Implementation (NSDI 24), Santa Clara, CA, USA, 2024, pp. 1285–1299.

[5] T. Julienne, “GLB: GitHub’s open source load balancer,” The GitHub Blog, Aug. 8, 2018. [Online]. Available: https://github.blog/engineering/infrastructure/glb-director-open-source-load-balancer/

[6] V. Olteanu, A. Agache, A. Voinescu, and C. Raiciu, “Stateless datacenter load-balancing with Beamer,” in Proc. 15th USENIX Symp. Networked Systems Design and Implementation (NSDI 18), Renton, WA, USA, 2018, pp. 125–139.

Appendix A: Team Work Distribution and Member Rating

[Appendices do not count toward the 10-page limit.]

A.1 Work Distribution

- *Ratings are agreed by the whole team and must be UNIQUE no two members may receive the same rating (no ties).*
- *Scale: 1–10 (10 = highest contribution). With 3 members, use three different values (e.g., 10, 9, 8).*

#	Member Name	Roll No.	Tasks Done	Rating (1–10, unique)
1	Akshit Chhabra	24110026	Led the project and directed the AI sessions. Chose the problem and the Unimog-style design, and the move from XDP to TC. Reviewed the code and the results. Ran the single-laptop experiments and the repeat runs. Report.	10
2	Bhavay Goyal	24110070	Read the Maglev, GLB and Prequal material. Checked the results tables and plots against the run data. Set up and ran the 4-laptop test. Slides.	8
3	Devarshi Savalgi	24110103	Read the material on measurement method. Tested the build and the unit and packet tests on a second machine. Set up and ran the 4-laptop test. Proofread the report.	7
4	Kunal Chandola	24110176	Read the Unimog and Beamer material and helped fix the design and the research questions. Reviewed the controller and redirector code. Set up and ran the 4-laptop test. Slides.	9

Appendix B: AI Usage Details

B.1 Prompts

Include prompts (or a link to the chat logs) for the most significant uses listed in Table.

#	AI Tool	Purpose
1	Claude Code (Claude Opus 5.5)	Research: kinds of load balancers, the papers, choosing the problem and the design (prompts 1–16). Prompts: https://github.com/akshit-codes/CS331-T020-MiniKatran/blob/main/ai/Prompts.pdf
2	Claude Code (Claude Opus 5.5)	Implementation: the eBPF director and redirector, the controller, the simulator and the tests (prompts 17–21). Prompts: https://github.com/akshit-codes/CS331-T020-MiniKatran/blob/main/ai/Prompts.pdf
3	Claude Code (Claude Opus 5.5)	Benchmark harness, running the experiments and the analysis (prompts 22–25). Prompts: https://github.com/akshit-codes/CS331-T020-MiniKatran/blob/main/ai/Prompts.pdf
4	Claude Code (Claude Opus 5.5)	Reviews of the code and results, fixes, the 4-laptop scripts and run, and ideas for the slides (prompts 26–38). Prompts: https://github.com/akshit-codes/CS331-T020-MiniKatran/blob/main/ai/Prompts.pdf
5	Claude Code	Writing this report from our results, section by section. Prompts: https://github.com/akshit-codes/CS331-T020-MiniKatran/blob/main/ai/Prompts-report.pdf

The prompts and the AI's replies are shared in the ai folder on GitHub:
https://github.com/akshit-codes/CS331-T020-MiniKatran/tree/main/ai